House Price Prediction Using Neural Networks

Graphs and Networks — Group Project Report

1. Problem Understanding

The primary objective of this project is to create a predictive model for Californian real estate pricing. We approach this using a supervised learning regression approach, aiming to predict a continuous target variable: the median house value for certain areas. Property prices are never influenced by isolation; they are significantly influenced by a combination of individual structural characteristics and the broader economic factors of their surrounding neighbourhoods. Our core task is to model these complex relationships using a standard artificial neural network. Furthermore, we seek to determine whether encoding spatial proximity into a simple graph structure can extract impactful input features to improve our model's baseline prediction accuracy.

2. Methodology

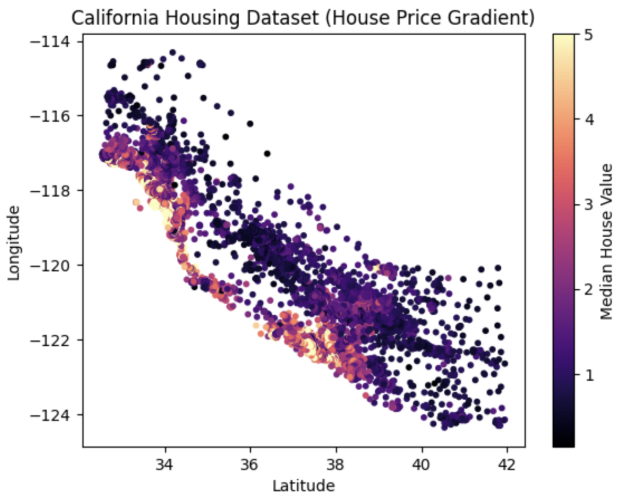
2.1 Dataset Description

To train and evaluate our models, we use the California Housing dataset, which is publicly accessible with the scikit-learn library. It is important to note that the data does not represent individual homes. Instead, each data point corresponds to a census block group, acting as a localised housing area in California.

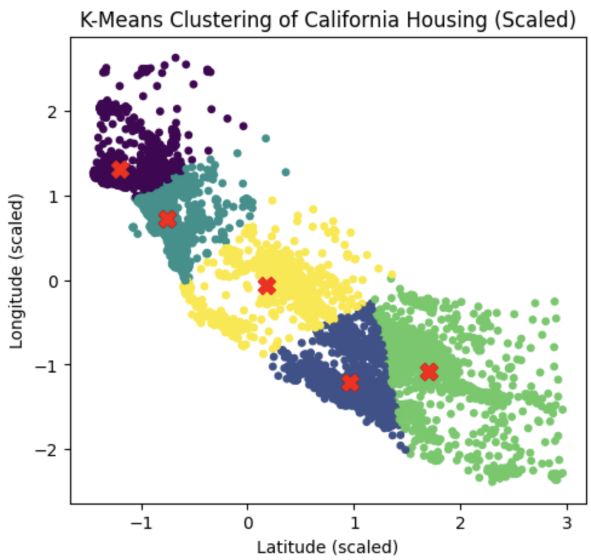
The dataset provides a comprehensive set of input features comprising aggregated housing and population statistics. These numerical metrics include:

- Median income of the residents.
- House Age, average number of rooms and bedrooms per household.
- General population-related statistics for the block group.
- Geographical location (latitude and longitude).

Initial analysis suggests these metrics all carry a significant role in the target house pricing. Using simple visualisation plots alongside clustering and heatmaps, it can better illustrate how impactful these metrics are.



(a) Initial price and income distributions.



(b) k-NN clusters: five main housing regions.

Nearest Neighbours Clusters (Figure 1b) show the main 5 housing areas in California (from left to right: Sacramento (rural north area), San Francisco Bay Area (incl. San Jose), Central Valley (Fresno centre), Los Angeles, San Diego (far south area)). Geographically this shows the north is much more rural with the smallest clusters (fewest houses) being in the Sacramento area. In the Centre cluster, you can see the inhabitable strip running through the centre (which would be the Great Valley, hence the lack of housing in that area). The most popular areas are visualised as the largest, more dense clusters of LA and San Diego regions, where housing is more dense and the populations are larger.

As an initial visualisation of the base-prediction housing prices, we can see the most expensive properties tend to be closer to the coastline overall, with a larger proportion of the most expensive housing in the Los Angeles area specifically (as expected).

You can see a similar gradient follows for average neighbourhood income as one of the 8 features in this dataset (Figure 2a). Majority of the higher incomes are in LA again and near the coast in the San Francisco area.

2.2 Data Preprocessing

In order to effectively explore the dataset, we utilised Pandas to manipulate the dataset. Sklearn has a prebuilt option to import this directly, allowing us to split the dataset by the target (median house price) and the input features.

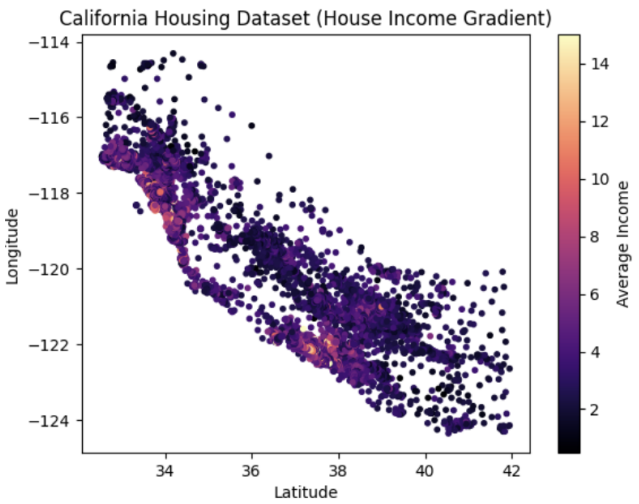
Subsequently, the data was split in order to effectively train the upcoming model. Selecting a low split risks underfitting (the model is too simple to be useful). Conversely, high split risks overfitting where the test size is too small for reliable evaluation. We decided to go for a standard 80:20 (Buhl, 2024) random split to balance training with evaluation. This split was randomised using a specific state value to keep results consistent.

Additionally, neural networks are sensitive to the magnitudes of the different metrics used. Using the raw data would cause the algorithm to handle various gradients, hindering convergence. To prevent this, we applied standard normalisation only to the input features. This allowed us to work with metrics that move on a uniform scale without impacting the train environment.

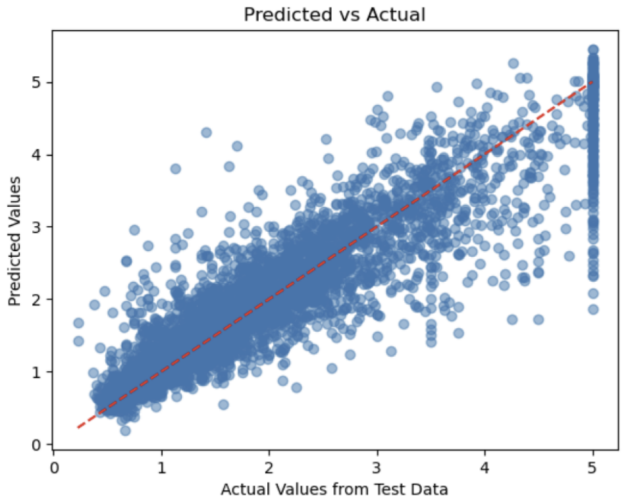
2.3 Baseline Neural Network

The `MLPRegressor` performs both forward propagation and back propagation to reduce the error between the predicted and actual outputs. The code uses two hidden layers, 64 and 32. These layers are the core components of the neural network's architecture. Each layer computes $z = WX + b$ where X is each input feature, W is the weight of each feature, and b is the bias. It forms linear combinations of the input variables in the dataset and then models the target as a non-linear activation function which is ReLU in this case as it is good at modelling non-linear relationships and house prices are not linear.

Finally, the metrics computed outside of the model (both R^2 test and mean squared error) compare the predicted output with the actual output to see how well the model performs. The R^2 test shows that the model is neither overfitting nor underfitting, therefore showing that the predicted outputs and actual outputs are similar and well balanced.



(a) Income gradient mirrors price map.



(b) Baseline actual vs. predicted prices.

The figure above (Figure 2b) is a plot of the predicted values against the actual values. The strong positive correlation following the red dashed line confirms what the R^2 test showed, which was that the model is well balanced. The Mean Squared Error computes the calculation $\text{MSE} = \frac{1}{n}\sum(y_{\text{actual}} - y_{\text{predicted}})^2$ which is a clear comparison of the predicted and actual outputs.

2.4 Constructing Neighbourhood-based Features

Feature 1: Average Neighbour House Price (`avg_nb_price`)

A neighbourhood based feature was added to capture spatial relationships between housing areas. This idea came from the assumption that geographically close areas tend to have similar house prices. Using the original latitude and longitude variables, neighbourhood connections were defined through a k-nearest neighbours (k=10) approach based on Euclidean distance.

To avoid data leakage, the k-Nearest Neighbours model was fitted only on training data coordinates, and the neighbours for both training and test observations were identified exclusively from the training set. Using these neighbour indices, a new feature representing the average house price of neighbouring areas (`avg_nb_price`) was computed from training target values.

This has resulted in two new variables, one for the train sample and another for the test sample, both of which were derived solely from training price information. `avg_nb_price` feature was added to capture local spatial price

patterns into the model. This allows the enhanced neural network to better learn geographical dependencies and improve prediction performance compared with the baseline model which only used original features.

Feature 2: Average Neighbour Median Income (`avg_nb_income`)

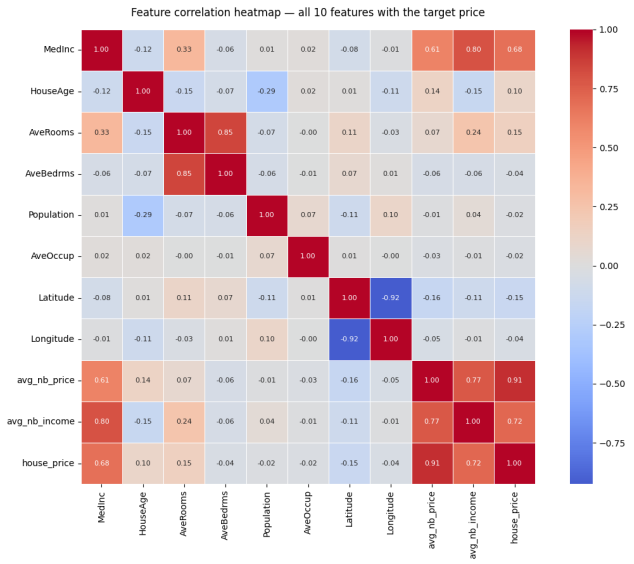
A second neighbourhood feature, `avg_nb_income`, was introduced to complement `avg_nb_price`. While `avg_nb_price` reflects nearby house values, `avg_nb_income` captures the earning level of residents, giving a different but related view of neighbourhood quality.

It was constructed using the same k nearest neighbours approach (k=10, Euclidean distance on latitude and longitude), so the same neighbour sets were used. For each point, the average income of its 10 nearest neighbours was calculated using training data only, ensuring no data leakage.

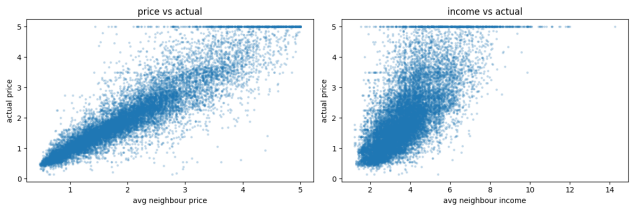
Both features are strongly correlated with house price (`avg_nb_price`: r=0.91, `avg_nb_income`: r=0.72) and also correlate with each other (0.77), indicating they are related but not redundant. Each captures a distinct aspect of the local area, so including both is justified.

The two features were then added to the dataset and scaled using a `StandardScaler` fitted on the training data, resulting in a final 10-feature input for Model 2.

Correlation Analysis



(a) Pearson correlation heatmap (all 10 features).



(b) Neighbourhood features vs. actual price.

The bottom row of the heatmap (Figure 3a) shows correlations with house price. `avg_nb_price` has the strongest correlation at 0.91, higher than median income at 0.68, making it the most informative feature. `avg_nb_income` is also strong at 0.72.

The two neighbourhood features correlate at 0.77, so they are related but not redundant. `avg_nb_price` reflects local house values, while `avg_nb_income` shows the economic profile of the area, meaning both add useful information.

In contrast, latitude and longitude have weak correlations with price at -0.15 and -0.04, showing that raw coordinates are far less informative than the neighbourhood features that we derived from them.

Zooming in on only our two additional features (Figure 3b), the above plots show both features correlate well with the actual house price (0.91 and 0.72) which confirms they're both useful. The difference in correlation values shows they're not just measuring the same thing and that income adds something extra on top of price.

Spatial Patterns

All three maps (Figure 4) show the same pattern. Expensive areas cluster along the coast, especially near the Bay Area and Los Angeles, while cheaper areas are inland. The `avg_nb_price` and `avg_nb_income` maps closely match the actual price map, which supports using neighbourhood features. It shows that location relative to nearby properties is strongly predictive of price, which is exactly what the graph-based approach captures. `avg_nb_income` has a slightly smoother gradient than `avg_nb_price`, reflecting its lower correlation with price, around 0.72 compared to 0.91. Still, it follows the same overall pattern, so it adds useful information rather than noise.

2.5 Model 2: Enhanced Neural Network

Model 2 is a feedforward neural network trained on `X_train_final` (10 features). To ensure a fair comparison with the baseline, the same starting architecture was used: two hidden layers (64 neurons → 32 neurons), ReLU activation, Adam optimiser, MSE loss function, and a maximum of 500 training iterations.

`GridSearchCV` with 3-fold cross-validation was also applied to optimise three hyperparameters, using the same search grid as the baseline model: `hidden_layer_sizes` ∈ {(64,32), (100,50), (32,16)}, `alpha` ∈ {0.0001, 0.001,

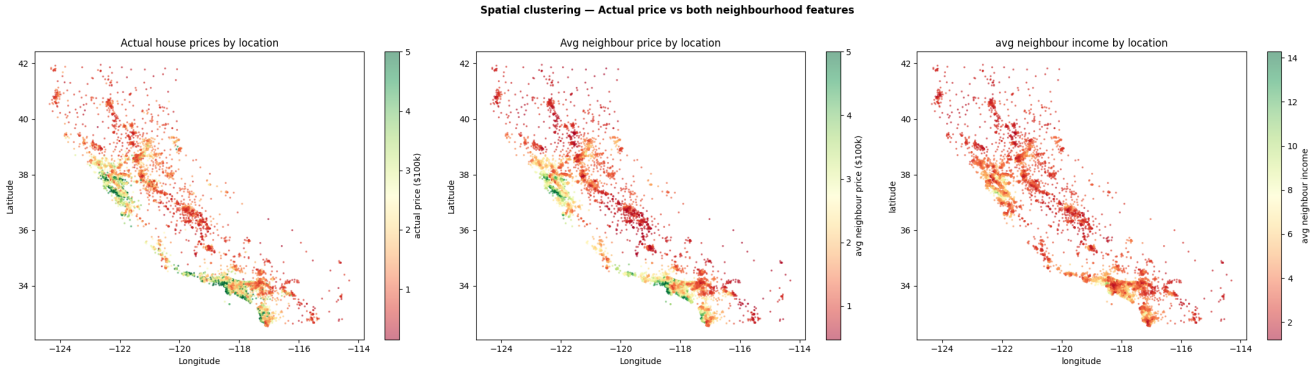


Figure 4: Geographic maps of actual price (left), `avg_nb_price` (centre), and `avg_nb_income` (right).

0.01}, `learning_rate_init` ∈ {0.001, 0.01}. Keeping the grid identical to the baseline means the only variable between the two models is the input feature set, making the comparison controlled and meaningful.

However, GridSearchCV made both models slightly worse on the held-out test set (baseline: 0.2569 → 0.2616; Model 2: 0.1681 → 0.1711), indicating mild overfitting to the 3-fold cross-validation splits. For a fair and transparent comparison, the default (untuned) configurations are therefore used as the primary result. Table 1 reports all four configurations for completeness.

3. Results

Table 1: Test set MSE across all model configurations. Primary comparison in bold.

Model	Features	Params	MSE	vs. Baseline
Baseline	**8 original**	**Default (64,32)**	**0.2569**	—
Baseline (GridSearch)	8 original	(100,50), α=0.0001	0.2616	+1.8%
Model 2	**10 features**	**Default (64,32)**	**0.1681**	**−34.6%**
Model 2 (GridSearch)	10 features	(32,16), α=0.0001, lr=0.01	0.1711	−33.4%

Model 2 achieves a test MSE of **0.1681** — a **34.6% reduction** over the baseline (0.2569). This substantial improvement is achieved purely by adding two neighbourhood features while keeping the same architecture. Notably, the initial Model 2 (before grid search) already achieved 0.1681, which is lower than the optimised baseline of 0.2616. This indicates that even without hyperparameter tuning, the neighbourhood features alone create a major performance gain. The fact that GridSearchCV made *both* models slightly worse confirms that the neighbourhood features — not hyperparameter tuning — are responsible for the improvement. Figure 5 illustrates the comparison.

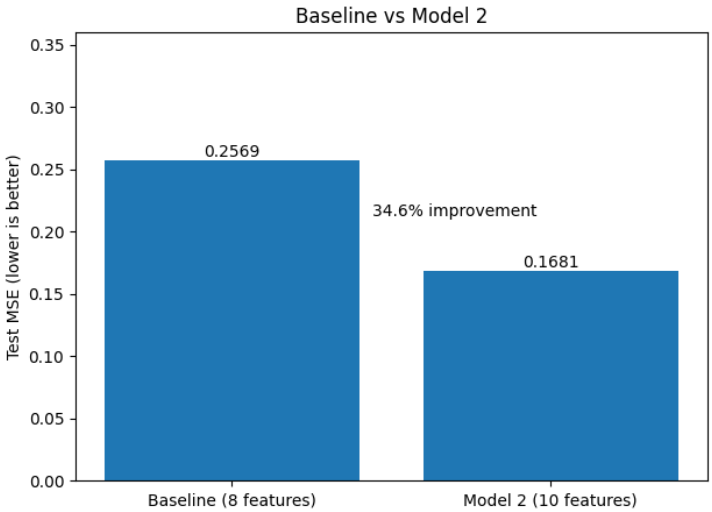


Figure 5: MSE comparison: baseline (0.2569) vs. Model 2 (0.1681), a 34.6% reduction.

4. Discussion

The results of the study shows that adding spatial neighbourhood features substantially optimises the predictive performance of the neural network models for housing price estimation. The baseline model which was trained by using only the original socioeconomic and housing features has achieved a sensible predictive accuracy. This confirms that the original dataset features, such as the median income of households and population size, can

provide useful data about local housing prices. However, the optimised model, which contained the two new neighbourhood-based features, resulted in a reduced prediction error. This highlights the importance of spatial dependency in real estate markets.

Introducing graph-derived features, namely the average neighbouring house price and average neighbouring income, assisted the neural network in how to capture local price dynamics that were not initially given in the original variables. There was a strong correlation observed between the derived features and the target variables, suggesting that nearby housing areas share similar economic and structural factors. This strongly aligns with economic rationale, as property markets are often affected by local demand, accessibility, and regional development patterns. As a result, the inclusion of neighbourhood features allowed the enhanced model to learn richer nonlinear relationships and improve its performance on unseen test data.

4.1 Limitations

Despite the improvement in prediction accuracy, there were two main limitations which should be acknowledged. First, the dataset was representing aggregation census block groups rather than individual house sales which reduces the model's ability to capture fine-scale variability in house prices. Second, the presence of a maximum limit placed on the value of house prices in the dataset has led to vertical clustering in the prediction plots, indicating that the model struggles to accurately estimate high-value properties. This is an indication that the neural networks may better benefit from alternative modelling strategies such as target transformation to better capture extreme values.

Additionally, Euclidean distance on raw coordinates ignores physical barriers and road networks; the fixed k=10 was not tuned and optimal neighbourhood size likely varies with housing density; and `avg_nb_price` is derived from the target variable itself, which may limit generalisability to markets where historical prices are unavailable.

4.2 Possible Improvements

- **Tune k:** include k in the hyperparameter grid to find the optimal neighbourhood size for different housing densities.
- **Distance-weighted aggregation:** give closer neighbours higher weight (e.g. inverse-distance) for more geographically precise features.
- **Target transformation:** log-scaling house prices before training would reduce the impact of the price ceiling on high-value properties.
- **Early stopping:** would prevent potential overfitting on longer runs and reduce training time.

5. Conclusion

In conclusion, this study demonstrates that incorporating simple graph-based spatial features alongside the original tabular housing variables can significantly enhance neural network regression performance. The results indicate that local spatial dependencies play an important role in housing price variation, and that even basic neighbourhood-based feature engineering can lead to meaningful improvements in model accuracy. Overall, the enhanced model provides a more accurate and robust framework for real estate price estimation, highlighting the practical value of integrating network-based approaches into supervised learning workflows.

References

Buhl, N. (2024). *Train, Validation, Test Split for Machine Learning.* Encord. https://encord.com/blog/train-val-test-split/

Pace, R.K. & Barry, R. (1997). Sparse Spatial Autoregressions. *Statistics and Probability Letters*, 33(3), pp.291–297.